

Data Structures: Assignment-5

Name: Suraj Shah

Admission Number: I24AI012

Q1: Write a C/C++ program to implement a circular linked list with the following operations:

- a) Insert an element at a specific position specified by the user.
- b) Insert an element at the beginning of the list
- c) Insert an element at the end of the list.
- d) Delete an element from a specific position specified by the user.
- e) Delete the first element from the list.
- f) Delete the last element from the list.

Note: The program should continuously prompt the user to select an operation and execute it until the user enters 0 to terminate.

Code:

```
#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int data;
    struct Node *next;
}Node;

void insertAtBeginning(Node** head, int data) {
    Node* newNode = (Node*)malloc(sizeof(Node));
    newNode->data = data;

    if (*head == NULL) {
        newNode->next = newNode; // Self-Loop
        *head = newNode;
    } else {
        Node* temp = *head;
        while (temp->next != *head) {
            temp = temp->next;
        }
        newNode->next = *head;
        temp->next = newNode;
    }
}
```

```

        *head = newNode;
    }
}

void insertAtPosition(Node** head, int data, int pos) {
    Node* newNode = (Node*)malloc(sizeof(Node));
    newNode->data = data;

    if (pos == 1) { // Insert at beginning
        insertAtBeginning(head, data);
        return;
    }

    Node* temp = *head;
    for (int i = 1; temp->next != *head && i < pos - 1; i++) {
        temp = temp->next;
    }

    newNode->next = temp->next;
    temp->next = newNode;
}

void insertAtEnd(Node** head, int data) {
    Node* newNode = (Node*)malloc(sizeof(Node));
    newNode->data = data;

    if (*head == NULL) {
        newNode->next = newNode;
        *head = newNode;
    } else {
        Node* temp = *head;
        while (temp->next != *head) {
            temp = temp->next;
        }
        temp->next = newNode;
        newNode->next = *head;
    }
}

void deleteFromBeginning(Node** head) {
    if (*head == NULL) return;

    Node* temp = *head;
    if (temp->next == *head) { // Only one node
        free(temp);
        *head = NULL;
        return;
    }

    Node* last = *head;

```

```

    while (last->next != *head) {
        last = last->next;
    }

    *head = (*head)->next;
    last->next = *head;
    free(temp);
}

void deleteAtPosition(Node** head, int pos) {
    if (*head == NULL) return;

    if (pos == 1) {
        deleteFromBeginning(head);
        return;
    }

    Node* temp = *head, *prev = NULL;
    for (int i = 1; temp->next != *head && i < pos; i++) {
        prev = temp;
        temp = temp->next;
    }

    if (temp == *head) return; // Invalid position
    prev->next = temp->next;
    free(temp);
}

void deleteFromEnd(Node** head) {
    if (*head == NULL) return;

    Node* temp = *head;
    if (temp->next == *head) { // Only one node
        free(temp);
        *head = NULL;
        return;
    }

    Node* prev = NULL;
    while (temp->next != *head) {
        prev = temp;
        temp = temp->next;
    }

    prev->next = *head;
    free(temp);
}

void displayList(struct Node* head) {
    if (head == NULL) {

```

```

        printf("List is empty.\n");
        return;
    }
    struct Node* temp = head->next;
    do {
        printf("%d -> ", temp->data);
        temp = temp->next;
    } while (temp != head->next);
    printf("(Repeat)\n");
}

int main() {
    Node *head=NULL;
    int choice, data, position;
    while (1) {
        printf("\nCircular Linked List Operations:\n");
        printf("1. Insert at specific position\n");
        printf("2. Insert at beginning\n");
        printf("3. Insert at end\n");
        printf("4. Delete from specific position\n");
        printf("5. Delete first element\n");
        printf("6. Delete last element\n");
        printf("7. Display list\n");
        printf("0. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);
        switch (choice) {
            case 1:
                printf("Enter data and position: ");
                scanf("%d %d", &data, &position);
                insertAtPosition(&head, data, position);
                break;
            case 2:
                printf("Enter data: ");
                scanf("%d", &data);
                insertAtBeginning(&head, data);
                break;
            case 3:
                printf("Enter data: ");
                scanf("%d", &data);
                insertAtEnd(&head, data);
                break;
            case 4:
                printf("Enter position: ");
                scanf("%d", &position);
                deleteAtPosition(&head, position);
                break;
            case 5:
                deleteFromBeginning(&head);
                break;

```

```

        case 6:
            deleteFromEnd(&head);
            break;
        case 7:
            displayList(head);
            break;
        case 0:
            exit(0);
        default:
            printf("Invalid choice!\n");
    }
}
}

```

Output:

<pre> Circular Linked List Operations: 1. Insert at specific position 2. Insert at beginning 3. Insert at end 4. Delete from specific position 5. Delete first element 6. Delete last element 7. Display list 0. Exit Enter your choice: 2 Enter data: 10 Circular Linked List Operations: 1. Insert at specific position 2. Insert at beginning 3. Insert at end 4. Delete from specific position 5. Delete first element 6. Delete last element 7. Display list 0. Exit Enter your choice: 3 Enter data: 20 Circular Linked List Operations: 1. Insert at specific position 2. Insert at beginning 3. Insert at end 4. Delete from specific position 5. Delete first element 6. Delete last element 7. Display list 0. Exit Enter your choice: 2 Enter data: 30 </pre>	<pre> Circular Linked List Operations: 1. Insert at specific position 2. Insert at beginning 3. Insert at end 4. Delete from specific position 5. Delete first element 6. Delete last element 7. Display list 0. Exit Enter your choice: 7 10 -> 20 -> 30 -> (Repeat) Circular Linked List Operations: 1. Insert at specific position 2. Insert at beginning 3. Insert at end 4. Delete from specific position 5. Delete first element 6. Delete last element 7. Display list 0. Exit Enter your choice: 5 Circular Linked List Operations: 1. Insert at specific position 2. Insert at beginning 3. Insert at end 4. Delete from specific position 5. Delete first element 6. Delete last element 7. Display list 0. Exit Enter your choice: 6 Circular Linked List Operations: 1. Insert at specific position </pre>
--	--

Q2: Write a C/C++ code to implement stack with following operations using array.

a) create () = Create a stack.

b) push() = Pushing (storing) an element on the stack

c) pop() = Removing (accessing) an element from the stack.

d) peek() = Get the top data element of the stack, without removing it

e) isFull() = Check if stack is full.

f) isEmpty() = Check whether the stack is empty, and return true or false.

Note: (i) Create a separate function for each of the operations defined above.

(ii) Define the stack as follows.

```
#define MAXSIZE 100
```

```
struct stack {
```

```
int stArr[MAXSIZE];
```

```
int top;
```

```
};
```

```
typedef struct stack STACK;
```

Code:

```
#include <stdio.h>

#define MAXSIZE 100
struct stack {
int stArr[MAXSIZE];
int top;
};
typedef struct stack STACK;

void create(STACK *s) {
    s->top=-1;
}

int isFull (STACK *s) {
    if (s->top==MAXSIZE-1) {
        return 1;
    }
}
```

```

    }
    return 0;
}

int isEmpty (STACK *s) {
    if (s->top==-1) {
        return 1;
    }
    return 0;
}

void push (STACK *s, int data) {
    if (isFull(s)) {
        printf("Stack Overflow");
        return;
    }
    s->top++;
    s->stArr[s->top]=data;
}

int pop (STACK *s) {
    if (isEmpty(s)) {
        printf("Stack Underflow");
        return -1;
    }
    int x=s->stArr[s->top];
    s->top--;
    printf("Element %d deleted\n",x);
    return x;
}

void peek (STACK *s) {
    if (isEmpty(s)) {
        printf("Stack Underflow");
        return;
    }
    printf("The top data element of the stack is %d",s->stArr[s->top]);
}

int main() {
    STACK A;
    int choice, data;
    create(&A);
    while (1) {
        printf("\nStack Operations:\n");
        printf("1. Push\n");
        printf("2. Pop\n");
        printf("3. Peek\n");
        printf("0. Exit\n");
        printf("Enter your choice: ");
    }
}

```

```

scanf("%d", &choice);
switch (choice) {
case 1:
    printf("Enter data: ");
    scanf("%d", &data);
    push(&A, data);
    break;
case 2:
    pop(&A);
    break;
case 3:
    peek(&A);
    break;
case 0:
    return 0;

default:
    printf("Invalid choice!\n");
}
}
}

```

Output:

<pre> Stack Operations: 1. Push 2. Pop 3. Peek 0. Exit Enter your choice: 1 Enter data: 10 Stack Operations: 1. Push 2. Pop 3. Peek 0. Exit Enter your choice: 1 Enter data: 20 Stack Operations: 1. Push 2. Pop 3. Peek 0. Exit Enter your choice: 1 Enter data: 30 Stack Operations: 1. Push 2. Pop 3. Peek 0. Exit Enter your choice: 2 Element 30 deleted Stack Operations: 1. Push 2. Pop 3. Peek </pre>	<pre> 0. Exit Enter your choice: 3 The top data element of the stack is 20 Stack Operations: 1. Push 2. Pop 3. Peek 0. Exit Enter your choice: 0 </pre>
---	---

Q3: Write a C/C++ code to implement stack with all the operations defined in

Q2 using Linked list.

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

typedef struct Node {
    int data;
    struct Node* next;
} Node;

typedef struct stack {
    Node* top;
} STACK;

void create(STACK **s) {
    *s = (STACK*)malloc(sizeof(STACK));
    if (*s == NULL) {
        printf("Memory allocation failed.\n");
        exit(1);
    }
    (*s)->top = NULL;
}

bool isEmpty(STACK *s) {
    return s->top == NULL;
}

void push(STACK **s, int data) {
    Node* newNode = (Node*)malloc(sizeof(Node));
    if (!newNode) {
        printf("Heap overflow. Cannot push %d\n", data);
        return;
    }
    newNode->data = data;
    newNode->next = (*s)->top;
    (*s)->top = newNode;
}

int pop(STACK **s) {
    if (isEmpty(*s)) {
        printf("Stack is empty. Cannot pop.\n");
        return -1;
    }
    Node* temp = (*s)->top;
    int popped = temp->data;
```

```

    (*s)->top = (*s)->top->next;
    free(temp);
    return popped;
}

int peek(STACK *s) {
    if (isEmpty(s)) {
        printf("Stack is empty. No top element.\n");
        return -1;
    }
    return s->top->data;
}

bool isFull(STACK *s) {
    return false;
}

int main() {
    STACK *A;
    int choice, data;
    create(&A);
    while (1) {
        printf("\nStack Operations:\n");
        printf("1. Push\n");
        printf("2. Pop\n");
        printf("3. Peek\n");
        printf("0. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);
        switch (choice) {
            case 1:
                printf("Enter data: ");
                scanf("%d", &data);
                push(&A, data);
                break;
            case 2:
                printf("Element %d deleted\n", pop(&A));
                break;
            case 3:
                printf("The top data element of the stack is %d", peek(A));
                break;
            case 0:
                return 0;

            default:
                printf("Invalid choice!\n");
        }
    }
}

```

Output:

```
Stack Operations:
```

```
1. Push
```

```
2. Pop
```

```
3. Peek
```

```
0. Exit
```

```
Enter your choice: 1
```

```
Enter data: 10
```

```
Stack Operations:
```

```
1. Push
```

```
2. Pop
```

```
3. Peek
```

```
0. Exit
```

```
Enter your choice: 1
```

```
Enter data: 20
```

```
Stack Operations:
```

```
1. Push
```

```
2. Pop
```

```
3. Peek
```

```
0. Exit
```

```
Enter your choice: 1
```

```
Enter data: 30
```

```
Stack Operations:
```

```
1. Push
```

```
2. Pop
```

```
3. Peek
```

```
0. Exit
```

```
Enter your choice: 2
```

```
Element 30 deleted
```

```
Stack Operations:
```

```
1. Push
```

```
2. Pop
```

```
3. Peek
```

```
0. Exit
```

```
Enter your choice: 3
```

```
The top data element of the stack is 20
```

```
Stack Operations:
```

```
1. Push
```

```
2. Pop
```

```
3. Peek
```

```
0. Exit
```

```
Enter your choice: 0
```